

Lecture 4: Quicksort

*Lecturer: Abraham Ladha**Scribe(s): Richard Zhang*

Previously, we have learned about the mergesort algorithm, a divide and conquer algorithm for sorting an array of values. In this lecture, we will show another divide and conquer sorting algorithm called quicksort.

1 What is Quicksort?

Quicksort works by repeatedly partitioning the current array into one left subarray and one right subarray where each element on the left subarray is less than or equal to some chosen **pivot** element and each element on the right subarray is greater than the pivot. Quicksort is then recursed on the half sized left and right subarrays, sorting them in place. Below is the pseudocode for Quicksort.

```
function quicksort(A, l, u)
  if l < u
    pivotIndex = partition(A, l, u)
    quicksort(A, l, pivotIndex - 1)
    quicksort(A, pivotIndex + 1, u)
quicksort(A, 1, n)
```

The `partition` function will perform the partition and return the index (`pivotIndex`) of the pivot element used. It has three jobs. It places the pivot element exactly in its correct index, it ensures everything below this is less than the pivot, and everything above this is greater than the pivot. The base case is implicit here: if the subarray $A[l \dots u]$ is of size zero or one ($l \geq u$), then the subarray is already sorted and we do not do anything.

1.1 Partitioning

The `partition` function works by selecting a pivot element in the array, and then moving the elements until the elements to the left of the pivot are less than or equal to the pivot and the elements to the right are greater than the pivot. Below is the pseudocode for the partition function that rearranges the subarray $A[l \dots u]$ and returns the index of the pivot.

```
function partition(A, l, u)
  pivot = A[u]
  i = l - 1
  for j in l..u-1
    if A[j] <= pivot
      i++
      swap A[i] and A[j]
```

```
    swap A[i+1] and A[u]
    return i + 1
```

The element $A[u]$ is chosen as the pivot and is used to partition the array. In the beginning of each iteration of the for loop (and at the end where we can think that $j = u$), the following is guaranteed:

- The indices from l to i have their elements less than or equal to the pivot.
- The indices from $i + 1$ to $j - 1$ have their elements greater than the pivot.

After the loop terminates, all the nonpivot elements will be partitioned such that the elements from indices l to i will be less than or equal to the pivot and the elements from indices $i + 1$ to $u - 1$ will be greater than the equal. We swap the pivot element with the leftmost element greater than pivot (which is at index $i + 1$) to put the pivot at the right place. Since the pivot is at $i + 1$, we return this index.

Below is an example with the array $[2, 8, 7, 1, 3, 5, 6, 4]$. Red indicates elements less than or equal to the pivot while blue indicates elements greater than the pivot.

Notes/partition example.png

The loop has $O(n)$ iterations and does $O(1)$ work at each iteration. Therefore, the runtime of the `partition` function is $O(n)$.

2 Quicksort Runtime Analysis

The runtime analysis for quicksort gets a little tricky since it is not always the case that the input is split equally among the subproblems. Let's consider the best case and worst case scenarios regarding how the input is split.

2.1 Best Case Analysis

The best case occurs when the partitioning leads to both subproblems of equal size (or that the sizes of the subproblems only differ by one). This happens when the median of the array (or number closest to the median) is selected as the pivot. If this always occurs, we can treat it as two subproblems with each of their inputs being halved and have the partitioning be the work done at each subproblem. This results in the following recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

We can apply the Master theorem and get that $T(n) = O(n \log n)$. This is unfortunately only under the assumption we always pick good pivots.

2.2 Worst Case Analysis

The worst case occurs when the partitioning leads to one subproblem with $n - 1$ elements and one with 0 elements. This happens when the chosen pivot is the maximum or minimum element of the array. Using the `partition` function defined in the previous section, this scenario will always occur when the array is already sorted. If this happens every time we partition, we get the following recurrence:

$$\begin{aligned} T(n) &= T(n - 1) + T(0) + O(n) \\ &= T(n - 1) + O(n) \text{ (Since } T(0) \text{ would be constant)} \end{aligned}$$

We can show that $T(n) = O(n^2)$ via two ways:

- **Induction:** We can show through induction that for all positive values of n , there exists a constant c such that $T(n) \leq cn^2$. For the base case where $n = 1$, this is trivial. For the induction case, suppose that $T(n - 1) \leq c(n - 1)^2$. We can substitute this in the recurrence and get the following:

$$\begin{aligned} T(n) &= T(n - 1) + O(n) \\ &\leq c(n - 1)^2 + dn \\ &= cn^2 - 2cn + c + dn \\ &\leq cn^2 + n^2 \\ &= (c + 1)n^2 \end{aligned}$$

Since $T(n) \leq (c + 1)n^2$, we can conclude that $T(n) = O(n^2)$ based on the definition of Big O.

- **Repeated Substitution:** We can repeatedly substitute the definition of the recurrence for smaller values (ex: $T(n - 1) = T(n - 2) + O(n - 1)$) until we reach the base case.

$$\begin{aligned} T(n) &= T(n - 1) + O(n) \\ T(n) &= T(n - 2) + O(n) + O(n - 1) \\ T(n) &= T(n - 3) + O(n) + O(n - 1) + O(n - 2) \\ T(n) &= O(n) + O(n - 1) + O(n - 2) + \cdots + O(1) \\ &= O\left(\frac{n(n + 1)}{2}\right) \\ &= O(n^2) \end{aligned}$$

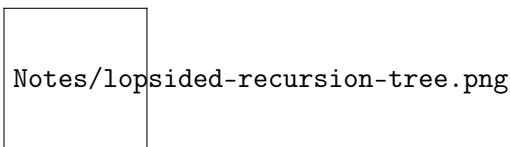
2.3 Average Case Intuition

The average runtime for quicksort is $O(n \log n)$. A more rigorous analysis for this will be shown later, but this section will provide some intuition.

Suppose that the partitioning of elements always produces a 999-1 split between the two subproblems, resulting in the following recurrence:

$$T(n) = T\left(\frac{999}{1000}n\right) + T\left(\frac{1}{1000}n\right) + cn$$

This seems really lopsided and not very optimal. But notice the pivot is not chosen a constant away from the maximum (or minimum). Its still chosen very close, but an amount which varies as a function of n . However, this $T(n)$ would still be $O(n \log n)$!



There would be $\log_{1000/999} n = O(\log n)$ levels in the recursion tree. At the first $\log_{1000} n$ levels of the recursion tree, cn work would be done. At the later levels, less than cn work would be done but it would still be upper bounded by cn . Therefore, the total amount of work done would be upper bounded by $cn \log_{1000/999} n$, which is $O(n \log n)$.

Additionally, the probability that the maximum element is always chosen decreases very quickly as n grows. Let we assume that the probability of each possible permutation of the elements is uniform. Without loss of generality assume the elements are distinct. There are $n!$ possible permutations, each equally likely. Then, we can also assume that each element in the array is equally likely to be at index u during the `partition` operation and thus be the pivot. The probability that the pivot value is the maximum value for one `partition` operation would be $\frac{1}{n}$. The probability that every partition operation would choose the maximum element during Quicksort would therefore be:

$$\left(\frac{1}{n}\right) \left(\frac{1}{n-1}\right) \left(\frac{1}{n-2}\right) \cdots \left(\frac{1}{1}\right) = \frac{1}{n!}$$

which would become very small as n grows large.

In the real world, not all permutations of the elements are equally probable. However, we can still have each element have an equal probability of being the pivot by random selection. The `partition` function would change in that an element from index l to u would be randomly selected as the pivot and be swapped with $A[u]$ before the partitioning begins. This allows quick sort to be robust and still be $O(n \log n)$ on average.

2.4 Average Case Analysis

With each element being chosen as a pivot with probability $\frac{1}{n}$, we can do a more rigorous analysis of the average runtime. If a chosen pivot element would be at index i in the sorted array (using zero-indexing), then the left sub problem would be of size i and the right sub problem would be of size $n-i+1$, which results in a recurrence of $T(n) = T(i) + T(n-i+1)$.

We compute our recurrence as an average over all possible choices of i . Below is the derivation showing that the average runtime $T(n)$ would be $O(n \log n)$.

$$T(n) = \frac{1}{n} \left[\sum_{i=0}^{n-1} [T(i) + T(n-i+1)] \right] + n$$

$$T(n) = \frac{1}{n} \left[\sum_{i=0}^{n-1} T(i) + \sum_{i=0}^{n-1} T(n-i+1) \right] + n$$

Notice the summations are simply a permutation of one another. So the summations are equal.

$$\sum_{i=0}^{n-1} T(i) = \sum_{i=0}^{n-1} T(n-i+1)$$

We substitute one summation for the other, multiply by n , then substitute n for $n-1$

$$T(n) = \frac{1}{n} \left[\sum_{i=0}^{n-1} T(i) + \sum_{i=0}^{n-1} T(n-i+1) \right] + n = 2 \left[\sum_{i=0}^{n-1} T(i) \right] + n$$

We multiply both sides by n and substitute n for $n-1$.

$$nT(n) = 2 \left[\sum_{i=0}^{n-1} T(i) \right] + n^2$$

$$(n-1)T(n-1) = 2 \left[\sum_{i=0}^{n-2} T(i) \right] + (n-1)^2$$

Take the difference of the previous two, and the sum will telescope out. We then simplify.

$$nT(n) - (n-1)T(n-1) = \left[2 \left[\sum_{i=0}^{n-1} T(i) \right] + n^2 \right] - \left[2 \left[\sum_{i=0}^{n-2} T(i) \right] + (n-1)^2 \right]$$

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2n - 1$$

$$nT(n) = (n+1)T(n-1) + 2n$$

Divide out by $n(n+1)$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2}{n+1}$$

Substitute n for $n - 1$ and compute $T(n - 1)$ as a function of $T(n - 2)$. Then substitute this into $T(n)$. Repeated substitution will make this no longer a recurrence.

$$\begin{aligned}\frac{T(n)}{n+1} &= \frac{T(n-2)}{n-1} + \frac{2}{n} + \frac{2}{n+1} \\ \frac{T(n)}{n+1} &= \frac{T(n-3)}{n-2} + \frac{2}{n-1} + \frac{2}{n} + \frac{2}{n+1} \\ &\dots \\ \frac{T(n)}{n+1} &= \frac{T(1)}{2} + 2 \sum_{i=3}^{n+1} \frac{1}{i}\end{aligned}$$

A simple proof by induction will show

$$\sum_{i=1}^{2n} \frac{1}{i} \leq 1 + n$$

. We approximate our sum by $O(\log n)$. You can also use the approximation of the Harmonic numbers.¹

$$\begin{aligned}\frac{T(n)}{n+1} &< C + 2 \log_2(n+1) \\ T(n) &< C(n+1) + 2(n+1) \log_2(n+1) \\ T(n) &= O(n \log n)\end{aligned}$$

¹https://en.wikipedia.org/wiki/Harmonic_number#Calculation